

Alumno: Diego Mora Márquez
Profesor: Luis de Jesús López Hernandez
Semestre 4

Mérida Yucatán **2026**

Aplicar Decimate y LOD a un Modelo

Optimización poligonal del activo «roca» entre Blender 5.2 y Unity 6 · Proyecto Cenote Vivo

1. Objetivo

Las rocas que rodean el agua del cenote son el activo con más detalle geométrico de la escena: cada una tiene casi diez mil triángulos para que su superficie irregular se vea natural de cerca. Sin embargo, la mayor parte del tiempo se observan desde lejos, ocupando apenas unos cuantos píxeles, y dibujar todos esos triángulos sería trabajo desperdiciado para la GPU de un teléfono. El objetivo de este ejercicio fue generar versiones simplificadas del modelo con el modificador **Decimate** de Blender y configurar en Unity un **LOD Group** que elija automáticamente la versión adecuada según el tamaño del objeto en pantalla.

2. Preparación del modelo en Blender

Se crearon dos variantes del activo, `roca` y `roca2`, para que la escena no repita la misma forma. Cada una parte de una icoesfera subdividida, deformada con dos modificadores *Displace* (textura Voronoi para el volumen general y Clouds para la rugosidad fina), aplanada en la base para que asiente en el suelo y llevada a exactamente **9 826 triángulos**, que constituyen el nivel de máximo detalle. Después se generaron coordenadas UV con *Smart UV Project* y se aplicó sombreado suave.

Antes de exportar se dejaron las mallas listas para el motor:

- **Escala aplicada** (`Apply Scale`): todas las mallas tienen escala (1, 1, 1), por lo que Unity no hereda factores que alteren la iluminación o la física.
- **Origen en la base del objeto**, centrado en X y Y, para colocar las rocas directamente sobre la plataforma.
- **Jerarquía limpia:** un objeto vacío `roca` como padre y los tres niveles como hijos con el sufijo `_LOD0`, `_LOD1` y `_LOD2`.

3. Metodología del modificador Decimate (Collapse)

El modo **Collapse** reduce la malla colapsando aristas: en cada paso elige la arista cuya eliminación produce el menor error geométrico, funde sus dos vértices en uno y elimina los triángulos que quedan degenerados. El parámetro *Ratio* indica la fracción de triángulos que debe conservarse. Como el algoritmo prioriza las aristas en zonas casi planas, primero desaparecen los detalles que

menos aportan a la forma y la **silueta perimetral se preserva** hasta reducciones bastante agresivas. Se activó *Triangulate* para que el resultado sea consistente con lo que recibe la GPU.

El proceso se automatizó con un script de Python (`Tools/blender/make_rocas.py`) ejecutado en Blender en modo *background*, de modo que el resultado es reproducible. El núcleo de la generación de niveles es:

Fragmento de make_rocas.py: creación de los niveles con Decimate (Collapse).

```
for lvl, ratio in {1: 0.45, 2: 0.25}.items():
    bpy.ops.object.duplicate()                  # copia de roca_LOD0
    lod = bpy.context.active_object
    lod.name = f"{name}_LOD{lvl}"
    dec = lod.modifiers.new("Decimate", 'DECIMATE')
    dec.decimate_type = 'COLLAPSE'
    dec.ratio = ratio                           # 0.45 → LOD1, 0.25 → LOD2
    dec.use_collapse_triangulate = True
    bpy.ops.object.modifier_apply(modifier=dec.name)
    lod.parent = parent                         # vacío "roca"
```

3.1 Valores de reducción poligonal

Tabla 1. Valores medidos después de aplicar el modificador (idénticos en roca y roca2).

Nivel	Ratio Decimate	Reducción (%)	Triángulos	Vértices (Blender)	Vértices (Unity)	Conserva silueta
LOD_0	1.00	100 % (Base)	9 826	4 915	7 380	Sí
LOD_1	0.45	45 %	4 420	2 212	3 751	Sí
LOD_2	0.25	25 %	2 456	1 230	2 286	Sí, con perfil suavizado

Los conteos de triángulos coinciden exactamente con los objetivos. La diferencia en el número de vértices entre Blender y Unity es normal: la GPU necesita un vértice distinto por cada combinación de posición, normal y coordenada UV, así que en las costuras de UV y en los bordes duros Unity duplica vértices al importar.

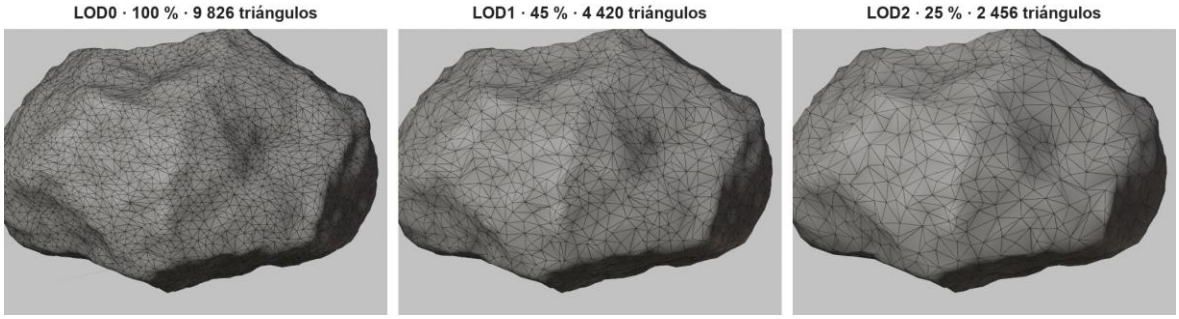


Figura 1. Malla alámbrica de los tres niveles de la roca en Blender (render Workbench con modificador Wireframe).

En la figura se aprecia cómo la densidad de la malla baja de forma notable de LOD0 a LOD2, mientras que el contorno de la roca —lo que el ojo percibe primero a distancia— se mantiene prácticamente igual. LOD2 pierde los relieves pequeños de la superficie, pero a la distancia en que se usa esos detalles ocupan menos de un píxel.

4. Exportación e importación

Ambas rocas se exportaron en un solo archivo `Assets/Models/Rocas.fbx` con *Apply Unit*, transformaciones horneadas (*Bake Space Transform*), eje adelante −Z y eje arriba Y, sin huesos auxiliares. En Unity se configuró el importador sin materiales embebidos (se asigna el material `M_Roca`, con mapa de normales generado desde la textura de roca del proyecto), con compresión de malla media y sin acceso de lectura en CPU, para que la malla ocupe memoria solo en la GPU.

5. Configuración del componente LOD Group en Unity

Para cada roca se creó un prefab (`Assets/CenoteVivo/Prefabs/roca.prefab` y `roca2.prefab`) con un componente **LOD Group** en el objeto padre. Unity decide el nivel comparando la altura del objeto en pantalla —su *Screen Relative Transition Height*— con los umbrales definidos:

Tabla 2. Umbrales del LOD Group.

Nivel	Malla	Activo entre	Sombras
LOD 0	roca_LOD0 (9 826 tris)	100 % y 32.5 % de la altura de pantalla	Proyecta
LOD 1	roca_LOD1 (4 420 tris)	32.5 % y 12.5 %	Proyecta
LOD 2	roca_LOD2 (2 456 tris)	12.5 % y 1.0 %	No proyecta
Culled	—	Menos de 1.0 %	—

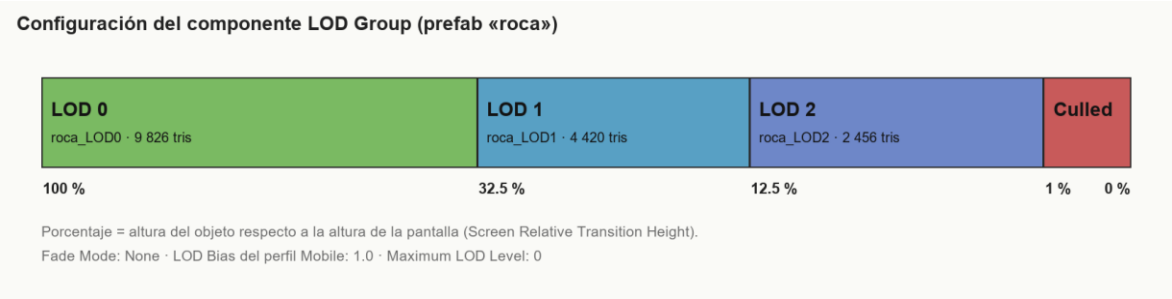


Figura 2. Esquema de la configuración del LOD Group del prefab «roca», con las mismas bandas que muestra el inspector de Unity.

Se tomaron tres decisiones adicionales. Primero, **LOD2 no proyecta sombras**: a esa distancia la sombra de una roca pequeña no se distingue, y omitirla evita dibujar la malla una vez más en el mapa de sombras. Segundo, se usó *Fade Mode: None*, porque el fundido cruzado entre niveles dibuja dos mallas a la vez durante la transición y añade costo de fragmentos. Tercero, el perfil de calidad **Mobile** mantiene un *LOD Bias* de 1.0, de modo que los cambios ocurren exactamente en los porcentajes configurados; el perfil de PC usa 2.0, que retrasa las transiciones para mostrar más detalle.

La configuración se aplica por código en `CenoteVivoSetup.cs`, lo que garantiza que ambos prefabs tengan los mismos valores:

Configuración del LOD Group (CenoteVivoSetup.cs).

```
var lods = new LOD[3];
float[] transitions = { 0.325f, 0.125f, 0.01f };
for (int i = 0; i < 3; i++)
{
    var child = go.transform.Find($"{name}_LOD{i}");
    lods[i] = new LOD(transitions[i], new[] { child.GetComponent<Renderer>() });
    if (i == 2) child.GetComponent<Renderer>().shadowCastingMode = ShadowCastingMode.Off;
}
group.SetLODs(lods);
group.fadeMode = LODFadeMode.None;
group.RecalculateBounds();
```

6. Evidencia del cambio de nivel en Unity

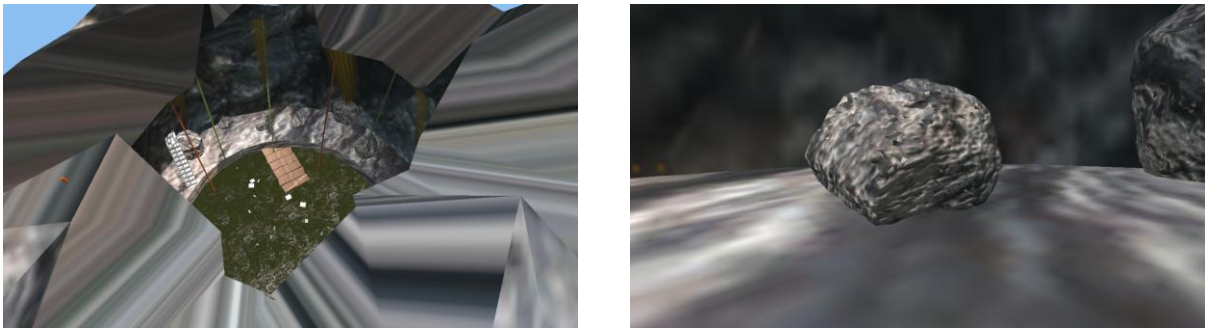
Para comprobar que el LOD Group funciona como se diseñó, la herramienta `CenoteCapture.cs` aisló una roca de la escena, colocó la cámara a la distancia exacta en que la roca ocupa un porcentaje dado de la pantalla y registró qué renderer quedaba visible:

Tabla 3. Verificación de las transiciones (roca_00, escala 0.74, perfil Mobile).

Altura en pantalla	Distancia de la cámara	LOD esperado	LOD activo (medido)
60 %	3.5 m	LOD 0	LOD 0
22 %	9.7 m	LOD 1	LOD 1
6 %	35.4 m	LOD 2	LOD 2
0.6 %	354.1 m	Culled	Culled



Figura 3. La misma roca renderizada en Unity a tres distancias con alambre superpuesto: la densidad de triángulos baja a medida que la roca se aleja.



Cámara alejada: rocas en LOD1–LOD2 **Primer plano: roca cercana en LOD0**

Figura 4. Las rocas colocadas alrededor del agua vistas desde las distancias usadas en el análisis de rendimiento.

7. Justificación del impacto en el rendimiento

Cada vértice de la escena pasa por el vertex shader en cada cuadro, y en la pasada de sombras se procesa otra vez. En una GPU móvil, con unidades de cómputo compartidas entre vértices y fragmentos y con un ancho de banda limitado para leer los búferes de geometría, reducir los vértices libera recursos para el sombreado de píxeles, que es donde se va la mayor parte del presupuesto.

Con 16 rocas en la escena, dibujarlas todas en LOD0 representaría unos 157 000 triángulos solo en rocas. Las mediciones del Profiler en la *development build* (ver Entregable 6) muestran el efecto real en la vista panorámica, donde todas las rocas quedan lejos:

Tabla 4. Efecto del LOD en la vista panorámica (incluye la pasada de sombras).

Escenario (vista panorámica)	Triángulos por cuadro	Vértices por cuadro	Tiempo de GPU
Sin LOD (LOD0 forzado) y sin batching	299 807	226 908	0.42 ms
Con LOD Group, SRP Batcher y static batching	39 411	40 564	0.36 ms
Reducción	**−86.9 %**	**−82.1 %**	**−14 %**

La reducción de triángulos es mayor que la proporción del Decimate porque, además de usar mallas más ligeras, las rocas en LOD2 dejan de dibujarse en el mapa de sombras. En el primer plano la diferencia desaparece (41 793 frente a 41 797 triángulos), como debe ser: la roca cercana necesita su máximo detalle y el sistema lo conserva. El tiempo de GPU de la tabla se midió en una GPU de escritorio, en la que la geometría pesa poco; en un teléfono el ahorro de vértices tiene un efecto proporcionalmente mayor, porque el procesamiento de geometría compite por las mismas unidades que el sombreado.

8. Conclusión

El modificador Decimate en modo Collapse permitió obtener, en segundos y de forma reproducible, dos versiones simplificadas de cada roca que conservan la silueta. Combinadas con un LOD Group bien calibrado, la escena dibuja el detalle completo solo donde el usuario puede apreciarlo. El resultado medido —casi 87 % menos triángulos en la vista general sin pérdida visible de calidad— confirma que el LOD es una de las optimizaciones con mejor relación costo-beneficio para una aplicación 3D en Android.